

Muslim: A Deployed Arabic Voice AI Platform for Grounded Islamic Knowledge

Yahya Mohamed Elnawasany
Independent Researcher, Egypt
yahyaalnwsany39@gmail.com

Abstract

We present MUSLIM, a production Arabic voice AI platform serving grounded, sourced Islamic knowledge to real users. Beyond a real-time voice pipeline (NeMo Arabic ASR, an OpenAI-compatible LLM endpoint, self-hosted TTS) and a deterministic multi-source retrieval layer routed across six Model Context Protocol servers, we report three things a research prototype typically lacks. First, a released family of fine-tuned Arabic Islamic model artifacts: an efficient tool-routing LLM (Muslim-6B-PRO, 5.94B parameters) and a Modern Standard Arabic TTS model (Fasih-TTS-V1) that ranks 5th of 17 overall and 2nd of 11 open-weight systems on the community-voted Arabic TTS Arena for MSA. Second, an account and metering layer – a free per-account turn allowance, capacity-aware refusal, and email verification deferred to the point it actually matters – that turns an open demo into an operable, abuse-resistant product. Third, a three-layer observability stack (liveness, error reporting, product analytics) built specifically around the system’s characteristic failure mode: a GPU-bound agent host going silent while the web tier keeps serving normally. We report real, measured latency and accuracy figures (98.4% recitation-validation accuracy on 124 cases; end-to-end voice latency of 0.9–1.7s) and discuss the concrete engineering trade-offs and limitations of running an Islamic-knowledge voice product in production.

1 Introduction

Commercial voice assistants handle Arabic poorly, answer Islamic questions from parametric memory rather than cited texts, and route every conversation through third-party cloud infrastructure. For a domain where a misattributed narration is a real harm, these are not minor gaps. MUSLIM addresses them with an architecture built around three principles: Arabic (including Quranic Arabic) as a first-class citizen, no Islamic content generated

without retrieval from a verified source, and the compute-heavy components of the pipeline running on infrastructure the operator controls rather than a third party.

This paper reports on MUSLIM as a *deployed* system, not a prototype. Our contributions:

1. A real-time Arabic voice pipeline integrating Arabic-specialized ASR, an env-configurable LLM endpoint, and self-hosted TTS, with a six-server tool-retrieval layer (§3).
2. A released family of fine-tuned Arabic Islamic model artifacts, including an MSA TTS model independently ranked on a community leaderboard (§4).
3. An account and metering layer – a free rolling turn allowance, a fail-open capacity wall, deferred email verification – that is, to our knowledge, undocumented in prior Islamic-voice-AI work because prior work did not need to survive real traffic (§5).
4. A three-layer observability design targeted at a specific, non-obvious deployed-system failure mode (§6).
5. An evaluation combining real measured latency/accuracy figures with reliability evidence from an automated test suite (§7).

2 Related Work

Commercial voice assistants (Siri, Google Assistant, Alexa) answer Islamic questions from general-purpose parametric memory, without source attribution and without any Arabic-specific speech tuning. Text-based Islamic reference sites (Quran.com, Sunnah.com) provide searchable, sourced access to the Qur’an and Hadith but no conversational voice interface and no recitation feedback. Prior computational work on Quranic recitation has focused primarily on acoustic phoneme assessment via forced alignment or end-to-end classification ([Radford](#)

et al., 2023), both of which require Tajweed-aware acoustic models or large labeled-error corpora that do not yet exist at production quality; MUSLIM’s validator instead operates on normalized ASR text, trading acoustic-level Tajweed detection for a deployable, deterministic pipeline with no training data requirement. On the generation side, retrieval-augmented generation (Lewis et al., 2020) is well established as a hallucination mitigation for open-domain QA; we apply it specifically to reference-addressable religious corpora, where the dominant query pattern names a specific verse or narration and a deterministic direct-lookup retriever is both simpler and more precise than approximate semantic search. To our knowledge, no prior published system combines real-time Arabic voice interaction, source-attributed multi-corpus retrieval, deterministic recitation validation, and a production account/metering/observability layer in one deployed platform.

3 System Architecture

MUSLIM is a microservices architecture: a WebRTC selective forwarding unit (LiveKit; LiveKit Inc., 2024b,a) carries audio between the browser and a stateless voice agent, which runs Silero VAD (Silero Team, 2021), an Arabic ASR model, an LLM, and TTS, and dispatches tool calls to knowledge servers over the Model Context Protocol (MCP; Anthropic, 2024) via FastMCP (jlowin, 2024). The frontend (Next.js; Vercel Inc., 2024) mints the only credential that lets a client reach the agent, and is where account/metering logic lives (§5). Figure 1 shows the complete system.

3.1 Voice Pipeline

ASR uses NVIDIA NeMo’s (Kuchaiev et al., 2019) Arabic FastConformer model (Rekesh et al., 2023), chosen over general-purpose alternatives because its training distribution includes formal and religious Arabic, which matters directly for Quranic vocabulary that general web-audio-trained models under-represent. VAD is preloaded once per worker process to eliminate cold-start latency, and the session uses preemptive generation – beginning LLM inference during the trailing-silence window VAD requires before confirming end-of-utterance – to overlap two otherwise-sequential latency sources. The LLM is reached through a single OpenAI-compatible interface, keeping the serving backend a two-variable configuration choice (endpoint URL,

model name) rather than a code dependency; two small startup patches correct tool-schema and tool-call-token serialization quirks specific to the current OpenAI-compatible backend, without modifying the agent SDK itself. TTS is selected the same way, defaulting to the self-hosted model in §4 with a cloud fallback available behind the same switch.

3.2 Deployment: Two Coexisting Topologies

The system supports two deployment modes on identical application code, differing only in *where* each service runs. A single-host, fully self-hosted mode runs every service in containers on one operator-controlled machine. Our own production deployment instead runs a *hybrid* topology: the web/account tier runs on a managed platform with a managed Postgres database, the WebRTC media layer runs on a managed real-time infrastructure provider, and the voice agent together with the GPU-dependent ASR/TTS services run on a single operator-controlled host that *dials out* to the media layer and accepts no inbound connections. This keeps the Arabic-specific, compute-heavy core of the system entirely under operator control – the part this paper’s contributions concern – while the web tier benefits from managed-platform availability. The trade-off, discussed in §7.4, is that the web tier and the voice agent can now fail independently: the website can be fully up while no conversation is possible.

3.3 Knowledge Retrieval: Six MCP Servers

Islamic knowledge is retrieved, never generated from parametric memory. Table 1 lists the six MCP servers the agent connects to; three run on operator-controlled infrastructure. Tafsir (Quranic exegesis) is served by direct JSON file lookup across eight classical Arabic books – no embedding model, no cold start, sub-millisecond latency, exact precision on direct-reference queries. Hadith retrieval, previously delegated to an external community service that proved unreliable in practice, is now served by a self-hosted server over 50,000+ narrations across 17 collections, removing a third-party dependency from the critical path. Quranic audio is never synthesized by TTS: a verse request bypasses the LLM/TTS branch entirely and streams an authentic human recitation, because no TTS system reliably reproduces Tajweed phonological rules and recitation is, in the tradition this system serves, a specialized discipline with abundant authentic recordings already available.

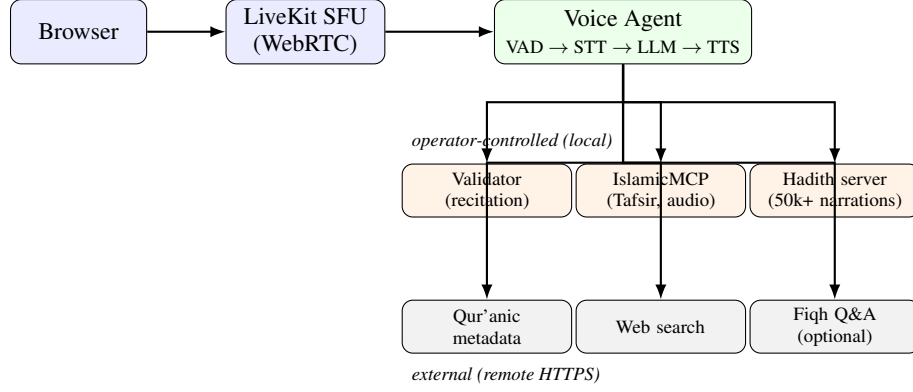


Figure 1: System overview. The browser connects over WebRTC to LiveKit; the stateless agent runs the voice pipeline and dispatches tool calls to six MCP servers (Table 1) – three operator-controlled, three external.

Server	Locus	Role
Validator	local	Recitation validation (§7)
IslamicMCP	local	Tafsir lookup, audio URLs
Hadith server	local	50k+ narrations, 17 collections
Qur'anic meta-data	remote	Revelation context, qira'at, morphology
Web search	remote	Contemporary topics, neural + keyword
Fiqh Q&A	remote	Scholarly Q&A corpus (optional)

Table 1: The six MCP servers the agent can call. Three run on operator-controlled infrastructure; three are external services reached over HTTPS with no operator data retained.

4 Released Model Family

Rather than depending solely on third-party inference, we release a family of fine-tuned Arabic Islamic model artifacts, available at <https://huggingface.co/NightPrince>. Table 2 summarizes the two primary releases.

We independently verified Fasih-TTS-V1’s standing on the Arabic TTS Arena (Navid AI, 2026), a community-voted, ELO-rated leaderboard, by retrieving its underlying results data directly rather than relying on a rendered page. As of our snapshot (2026-08-18; 245 head-to-head MSA battles), Fasih-TTS-V1 ranks **5th of 17** systems overall on Modern Standard Arabic and **2nd of 11** among open-weight systems, ahead of several commercial entrants. Because this is a live, continuously updated arena, we report the snapshot date and battle count rather than treating the rank as fixed. Two further fine-tuned Arabic ASR checkpoints (a Quran-recitation-specialized Fast-Conformer model and a diacritization-aware NeMo

Model	Summary
Muslim-6B-PRO	QLoRA fine-tune (4-bit NF4, $r=16$) of a 5.94B-parameter Qwen3-architecture base (Qwen Team, 2025), 262K context, on 2,731 curated tool-calling / production-derived examples via TRL (von Werra et al., 2020). Built for tool-call routing (31 tools) and persona discipline, not unsupported scripture recitation. Apache-2.0 (NightPrince, 2026b).
Fasih-TTS-V1	Fine-tune of Coqui XTTS v2 (Casanova et al., 2024) on 1,297 curated single-speaker MSA clips (~ 2.4 h), with a custom Arabic front-end (normalization, diacritization, chunking). CER 1.3% (vs. 1.8% human-recording baseline) on the SILMA Open-Source Arabic TTS Benchmark (SILMA AI, 2026), ranking highest for intelligibility among open-source systems tested. Coqui Public Model License (non-commercial) (NightPrince, 2026a).

Table 2: The two primary released model artifacts. Both are used in the production pipeline behind an environment-switchable configuration; the LLM currently serving live conversation traffic is a larger general-purpose model reached through the same OpenAI-compatible interface, with Muslim-6B-PRO as the path toward fully self-hosted inference.

model) round out the released family but are not evaluated further in this paper.

5 Accounts, Metering, and the Capacity Wall

A single server route mints the only credential that lets a client join a session with the agent, and is therefore where every access-control decision is concentrated. It resolves the caller’s session, checks their entitlement, and either refuses with one of six typed reasons (`no_session`,

signin_required, quota_exhausted, verify_email_required, at_capacity, agent_offline) or signs the allowance directly into the issued token as participant attributes the client cannot forge.

Tiers. An account is the only way into a conversation: there is no guest tier and no turns before sign-up. The service is free to every user and there is no paid tier: the single remaining allowance grants 30 turns on a *rolling 24-hour* window, a ceiling set by the fixed GPU capacity the deployment runs on rather than by a pricing model. This is a reversal. The system shipped with an anonymous tier carrying a small lifetime allowance, which acted as a preview and a sign-in wall, and it was retired in September 2026 after the free-allowance question was settled the other way. The migration is the part worth reporting: guest rows were kept rather than deleted, because their conversations carry captured turns that a cascading delete would have taken with them, and the session layer instead reads a surviving guest cookie as signed out. Enforcement is layered: the agent itself counts turns against the limit signed into its token and closes the session on reaching it – a hard stop that holds even if every usage report to the database fails – while the database accumulates cross-session usage for the rolling window, counted by turn index so a retrieved report never double-counts a user.

Deferred verification. Email confirmation is required to *refill* the allowance, not to sign up. Sign-up and first use are the highest-intent moment a visitor has, and putting a clicked email link between that intent and the product spends it; withholding only the refill defers the cost to a point where the user already knows whether the product is worth the click. An unverified account is therefore worth exactly one window’s turns. Retiring the guest tier widened this rather than narrowing it: with no lifetime allowance left, the grace period is now the first wall every user meets rather than the second.

Capacity wall. Because the GPU-dependent services run on a fixed-size host, the token route also checks the agent’s most recently reported active-job count before issuing a token, refusing with a distinct reason when saturated rather than letting the user experience an unexplained failed join – the two are otherwise indistinguishable from the user’s side. This check is deliberately *fail-open*: if the capacity query itself errors, requests are allowed through, so a monitoring fault cannot itself take the product down.

6 Observability

MUSLIM’s characteristic failure mode defeats naive monitoring: the GPU-bound agent host can go silent while the web application keeps serving 200s normally, so pinging the website learns nothing. We address this with a heartbeat that travels *outward* from the agent on a fixed interval to a judge that sits outside both halves of the system; staleness alone reveals an outage, with no crash-detection logic required. A public health endpoint encodes a three-way verdict (healthy / degraded – notably, alive but not registered with the media layer, invisible to a naive check / down) in both its body and HTTP status.

This liveness signal is deliberately one of three independent layers. Error reporting captures both web-tier exceptions and agent-side failures the agent deliberately swallows to keep a conversation alive – exactly the failures a healthy heartbeat cannot reveal. Product analytics tracks a deliberately small set of funnel events (session start, refusal reason, sign-up, verification, auth failure, and support contact), counting a conversation as started only when a token is actually issued rather than when a button is pressed, since the button is also pressed by users about to be refused. Privacy choices are consistent across all three layers: no session replay, no default PII capture, and analytics identity keyed to an internal id, never an email address.

7 Evaluation

7.1 Recitation Validator

The deterministic recitation validator – a seven-step Arabic normalization pipeline followed by a four-layer verse-matching search, with no LLM involvement – scores **98.4%** (122/124) on the 124-case suite released with it. The validator, its 2,290-pair Uthmani-to-Standard word mapping, and that evaluation are the subject of a companion paper (Elnawasany, 2026), which we cite rather than restate; the figures here are quoted from the released artifact and reproduce by running it.

Two results from that work bear on this system. First, both failures share one mechanism – a single substitution error can make a *different* verse a literal exact match, which is a property of the search design rather than of the morphological data. Second, a corpus-wide census finds that 16.5% of the Qur’an’s 6,236 verses (369 groups) share an ambiguous four-word normalized opening with at least one other verse; the remaining 83.5% resolve

unambiguously. The ambiguity is a property of the text, not a pipeline defect, and it is the reason the product asks for continuation rather than guessing between candidates.

One measurement result is worth reporting at this venue in its own right, because it is an operational failure rather than a modeling one. An earlier figure of 99.2% for this same validator was obtained in an environment where an optional fuzzy-matching dependency happened to be absent. The code logged a warning and returned an empty candidate list, so the fourth search layer was silently skipped rather than failing, and the evaluation scored a three-layer system while the documentation described four. The released artifact now depends on nothing outside the Python standard library, which removes the class of error rather than the instance: an accuracy number that moves with which packages are installed is not a property of the system being measured.

7.2 Latency

Measured directly on real data: ASR mean latency 235.5ms (RTF 0.025, i.e. real audio is transcribed roughly 40× faster than it plays), LLM first-token mean latency 519.7ms. Estimated end-to-end latency from speech end to first audio response is 0.9–1.7s under typical conditions – competitive with commercial assistants despite running the ASR/TTS stages on operator-controlled rather than hyperscale infrastructure.

7.3 Retrieval Grounding

On 30 held-out Islamic questions, retrieval succeeds for 100% of queries, and the retrieval-augmented condition adds only ~50ms mean latency over answering from parametric memory alone (786ms vs. 736ms) – confirming that grounding, which is the paper’s central integrity claim, is not purchased at a meaningful latency cost.

7.4 Reliability Evidence

Beyond point-in-time accuracy, an automated suite of over 1,000 tests (889 Python, 119 TypeScript) covering the validator, retrieval tools, and account/metering logic runs on a pre-push git hook mirroring a configured continuous-integration workflow; the CI workflow itself is currently inactive pending platform billing resolution on our private repository, a mundane operational constraint we report because it is the kind of practical deployment friction this venue specifically solicits, not

because we consider it resolved.

Limitations

Single-host availability. The hybrid topology (§3.2) means the web tier and account system remain available independently of the voice agent, but no conversation is possible while the single GPU host is offline – acceptable during beta, not for a paying userbase without redundant capacity. **Dialectal Arabic.** The ASR model targets Modern Standard and Quranic Arabic; dialectal speakers may see elevated error rates. **No formal user study.** All results are technical (accuracy, latency, retrieval success); no controlled study of learning outcomes or perceived response quality has been conducted. **RAG coverage boundary.** Questions outside the indexed Tafsir/Hadith/metadata corpora fall back to the LLM’s parametric knowledge and may be inaccurate; a formal faithfulness evaluation (e.g. RAGAS; [Es et al., 2023](#)) against expert-annotated ground truth is future work. **Model-family evaluation.** Muslim-6B-PRO’s model card does not yet report quantified benchmark results; we describe its architecture and training honestly rather than assert an accuracy figure it does not have. **Live-leaderboard citation.** The Arabic TTS Arena rank reported in §4 reflects a snapshot of a continuously updated, community-voted system and may shift.

Ethical Considerations

The system captures voice conversations for service improvement; consent is signed into the same access token that carries identity, so recording honors a user’s choice independent of whether usage metering is enabled, and an operator-level switch and the user’s own choice are both required for capture to occur. No session replay is enabled anywhere in the stack, and product-analytics identity is keyed to an internal identifier, never an email address – a deliberate choice given that a compromised analytics record for this product is a record of someone’s private religious inquiry. Because Islamic knowledge questions carry real consequences when answered wrongly, every retrieval path is designed to make the system decline or hedge rather than answer confidently from unverified parametric memory outside the domains explicitly covered by the retrieval layer (§7.4).

Acknowledgments

The author thanks Prof. Marwa Seddiq for supervision and guidance throughout this project.

References

- Anthropic. 2024. Model context protocol specification. <https://modelcontextprotocol.io>.
- Edresson Casanova, Eren Gölge, Erika Lima, Wagner de Sousa Jr, Kelly Aljafari, et al. 2024. XTTS: a massively multilingual zero-shot text-to-speech model. *arXiv preprint arXiv:2406.04904*.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.
- Yahya Mohamed Elnawasany. 2026. [A corpus-aligned Uthmani-to-Standard Quranic word mapping and a deterministic recitation validator](#). *Preprint*, arXiv:2609.14967.
- Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.
- jlowin. 2024. FastMCP: A python framework for the model context protocol. <https://github.com/jlowin/fastmcp>.
- Oleksii Kuchaiev, Jason Li, Huyen Nguyen, Oleksii Hrinchuk, Ryan Leary, Boris Ginsburg, Samuel Kriman, Stanislav Beliaev, Vitaly Lavrukhin, Jack Cook, Patrice Castonguay, Mariya Popova, Jocelyn Huang, and Jonathan M. Cohen. 2019. NeMo: A toolkit for building AI applications using neural modules. In *NeurIPS Workshop on Systems for Machine Learning*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.
- LiveKit Inc. 2024a. LiveKit Agents: Framework for building real-time AI voice agents. <https://docs.livekit.io/agents/>.
- LiveKit Inc. 2024b. LiveKit: Open source real-time communication platform. <https://livekit.io>.
- Navid AI. 2026. Arabic TTS arena: A community-driven leaderboard for arabic speech synthesis. <https://huggingface.co/spaces/Navid-AI/Arabic-TTS-Arena>. Live leaderboard, snapshot 2026-08-18.
- NightPrince. 2026a. Fasih-tts-v1: A modern standard arabic text-to-speech model. <https://huggingface.co/NightPrince/Fasih-TTS-V1>. HuggingFace model card. Accessed 2026-08-18.
- NightPrince. 2026b. Muslim-6b-PRO: A tool-routing fine-tune for islamic voice assistance. <https://huggingface.co/NightPrince/Muslim-6B-PRO>. HuggingFace model card. Accessed 2026-08-18.
- Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 28492–28518. PMLR.
- Dima Rekesh, Nithin Rao Koluguri, Samuel Kriman, Somshubra Majumdar, Vahid Noroozi, He Huang, Oleksii Hrinchuk, Krishna Puvvada, Ankur Kumar, Jagadeesh Balam, and Boris Ginsburg. 2023. Fast conformer with linearly scalable attention for efficient speech recognition. In *Proceedings of the IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*.
- Silero Team. 2021. Silero VAD: Pre-trained enterprise-grade voice activity detector. <https://github.com/snakers4/silero-vad>.
- SILMA AI. 2026. SILMA open-source arabic TTS benchmark. <https://huggingface.co/blog/silma-ai/arabic-tts-benchmark>.
- Vercel Inc. 2024. Next.js: The React framework. <https://nextjs.org>.
- Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. 2020. TRL: Transformer reinforcement learning. <https://github.com/huggingface/trl>.